

An Event-Driven Data Pipeline Using APIs, Kafka, Spark, and AWS

Welcome back! In our previous article, we built a batch **data pipeline** using Docker, Airflow, dbt, and Postgres – a pipeline that ran on a schedule and delivered data for analysis each day. Now, we’re taking a big step forward into real-time processing. In this tutorial, we’ll build an **event-driven (streaming) data pipeline** using **APIs, Apache Kafka, Apache Spark**, and **AWS**. Don’t worry if those terms sound new – we’ll break down every core concept as we go. By the end, you’ll understand how data can flow continuously from a source API through Kafka and Spark, landing in Amazon S3 for downstream use, all while using Docker to simulate the setup locally and discussing how to transition to AWS in production.

This is a comprehensive, beginner-friendly walkthrough. We’ll explain **everything** – what Kafka is (and what partitions and offsets mean), how Spark Structured Streaming processes data in **micro-batches**, why we use S3 for storage, and how we can orchestrate the whole pipeline. The goal is to show you a pipeline *like you’d find in real production*, but in a way that’s easy to follow for someone new to streaming data.

Let’s dive in! (Feel free to jump to any section using the headings.)

Pipeline Overview

Before getting into the tools, let’s clarify what our pipeline will do from end to end. Unlike the previous batch pipeline (which ran in periodic intervals), this **event-driven pipeline** will handle data continuously as it arrives. Here’s the high-level workflow:

- **Data Generation (API Producer):** A small app continuously fetches data from an API (e.g. a public “random user” API) and publishes these data events to Kafka. This simulates live events streaming into the system.
- **Streaming Ingestion with Kafka:** **Apache Kafka** acts as a **message broker** – a durable buffer that decouples the producing side from the consuming side. The incoming API data is written into a Kafka **topic** (think of a topic as a named feed or log for events).
- **Real-Time Processing with Spark:** **Apache Spark** (Structured Streaming) consumes the events from the Kafka topic in near real-time. Spark processes the data (for example, parsing JSON, filtering or aggregating fields) in small batches (micro-batches) and prepares it for storage.
- **Storage to Data Lake (Amazon S3):** The processed data is continuously written out to storage – in this case, an Amazon S3 bucket (cloud object storage). S3 will hold our curated data in files (e.g. Parquet or JSON), which can later be analyzed or loaded into a data warehouse.
- **Orchestration & Monitoring:** We use **Docker** to run all these components locally. In a production cloud environment, we’d use managed services (Amazon MSK for Kafka, Amazon EMR or Glue for Spark, etc.) and a tool to orchestrate and monitor the pipeline (for example, Apache Airflow or AWS Step Functions) to ensure each piece is running and connected. We’ll discuss how each component hands off to the next and how to keep the pipeline running reliably.

(Imagine a pipeline where an API feed produces events -> Kafka buffers and distributes those events -> Spark consumes and transforms them -> and S3 stores the outputs. Each component plays a distinct role, which we'll detail below.)

From Batches to Real-Time: Why Event-Driven?

Before we get hands-on, let's quickly contrast batch vs. streaming pipelines. In a batch pipeline (like our last one), data was collected and processed in chunks (e.g. once per day or hour). This introduces natural delays. In many modern scenarios – imagine tracking user activity on a website or IoT sensor readings – we need data *right away*. That's where an **event-driven architecture** shines.

- **Event-driven pipeline** means the system reacts to each new piece of data (event) as it happens. There isn't a "wait until tomorrow's schedule" – data flows continuously.
- This allows for **real-time analytics** and timely insights. For example, you could detect anomalies within minutes or update dashboards continuously.
- The trade-off is complexity: we need components that can handle continuous input and never-ending processing. We also need to design for resiliency, since a streaming job might run for weeks or months nonstop.

In our pipeline, Kafka and Spark are the backbone enabling this real-time flow. Let's introduce these technologies in an accessible way.

The Tech Stack (Tools Used)

We will use a few key technologies in this pipeline, each with a specific role:

- **Apache Kafka** – A distributed **event streaming platform**. Kafka is like a durable queue, but with superpowers: it can handle high volumes of events, store them reliably, and allow multiple consumers to read streams of events independently. We'll use Kafka to ingest and buffer the API data. *(We'll explain Kafka's internals – brokers, topics, partitions, offsets – as we proceed.)*
- **Apache Spark (Structured Streaming)** – A unified analytics engine for big data processing. Spark's Structured Streaming library allows writing streaming computations that look like batch queries. Spark will consume data from Kafka, process it (transforming or aggregating it), and output results continuously to S3. *(We'll dive into how Spark's micro-batch processing works and why it's beginner-friendly for streaming.)*
- **Docker** – We use Docker containers to emulate our production environment locally. Docker Compose will spin up services for Kafka (and its dependency Zookeeper), possibly an Airflow scheduler for orchestration, and a Spark environment. This way, everything runs on our machine but isolated in containers, similar to how they'd run on separate servers in production. **Docker is the glue** that ties our pipeline components together in development.
- **Amazon S3** – Amazon Simple Storage Service (S3) will serve as the **data lake** storage. It's a durable, scalable object store. In our local setup, we might just write to a local filesystem path to simulate S3, but in production we'll use a real S3 bucket. Storing data in S3 decouples storage from processing – once data lands in S3, various tools (Athena, Redshift, etc.) can use it for analysis.
- **Orchestration Tool (Airflow or similar)** – In streaming pipelines, orchestration is a bit different from batch jobs. We aren't scheduling a job to run and then stop; instead, we ensure continuous processes keep running. We'll use **Apache Airflow** in our example to schedule the data producer script and manage any batch tasks (like launching the Spark job if needed). In production, you might use Airflow (perhaps with a sensor to monitor the streaming job) or a service like AWS

Step Functions to coordinate processes. We'll discuss how we keep the pipeline running and recover from failures.

Local vs. Production: Locally, we'll run Kafka, Spark, etc. via Docker. In AWS, each component maps to a managed service for scalability:

- Kafka → **Amazon MSK** (Managed Streaming for Kafka) or Amazon Kinesis (an alternative streaming service).
- Spark → **Amazon EMR** (Elastic MapReduce) or AWS Glue (managed Spark ETL) for running Spark streaming applications on a cluster.
- S3 (storage) → still **Amazon S3** in AWS (just a real bucket).
- Airflow → **Amazon MWAA** (Managed Workflows for Apache Airflow) or AWS Step Functions / AWS Lambda for orchestrating and reacting to events.
- Docker → In production, instead of Docker Compose, we deploy services on cloud infrastructure (containers on ECS/EKS or just use the managed services directly).

Now that we know what we're using, let's set up our environment and then walk through building each part of the pipeline.

Setting Up the Environment with Docker Compose

In a real deployment, Kafka and Spark would run on separate servers or cloud services. For learning purposes, Docker Compose lets us run everything on one machine in isolated containers. It saves us from installing Kafka or Spark manually, and ensures all services can find each other via a virtual network.

Why Docker? Installing Kafka, Zookeeper, and Spark on your local machine directly can get messy (each has its own dependencies, configurations, and even specific Java versions). Docker lets us use pre-built images that contain everything needed for each service, and we can start or tear down the whole stack with one command. It also mirrors how you might deploy with containers in production.

Let's create a `docker-compose.yml` file to define our services. At minimum, we need:

- **Zookeeper:** Kafka's coordination service (Kafka relies on Zookeeper to manage brokers and topics metadata).
- **Kafka broker:** The Kafka server that will host our topics and handle message streaming.
- **Spark:** We can use a Spark container (there are Docker images that have Spark installed). For simplicity, we might run Spark in "local" mode inside a container, or use Spark's standalone cluster with a master and worker.
- **(Optional) Airflow:** If we use Airflow to schedule the API calls, we'd include an Airflow scheduler & webserver, plus a metadata database (Postgres) in the compose file.
- **(Optional) Local S3 emulator:** In case we want to simulate S3, we could include something like LocalStack or MinIO. But to keep things simple, we might just write to a local folder and treat it as our "S3".

For brevity, we won't show the entire Docker Compose file here, but let's look at a snippet to see how one of the services is defined:

```
services:
  zookeeper :
```

```

image: confluentinc/cp-zookeeper:7.4.0
container_name: zookeeper
ports:
  - "2181:2181" # Zookeeper listens on 2181
environment:
  ZOOKEEPER_CLIENT_PORT: 2181
  ZOOKEEPER_TICK_TIME: 2000

broker:
  image: confluentinc/cp-kafka:7.4.0
  container_name: broker
  depends_on:
    zookeeper:
      condition: service_healthy
  ports:
    - "9092:9092" # Kafka broker exposed on 9092
  environment:
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://broker:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_NUM_PARTITIONS: 3
    # (other Kafka configs omitted for brevity)

```

In this snippet, we define a Zookeeper service and a Kafka broker service. The Kafka broker's config points to Zookeeper and sets some basic parameters: it's listening on port 9092, and we set a default replication factor and number of partitions. (Replication factor 1 means no extra copies of data, fine for dev; in production you'd use 3 for reliability.) We also expose Kafka's port so our client apps can connect.

When we run `docker-compose up -d`, Docker will pull these images and start the containers. We'll have a running Kafka broker ready to accept connections.

Tip: To verify Kafka is up, you can use Kafka's command-line tools inside the container. For example, list topics with:

```
docker exec broker kafka-topics --bootstrap-server localhost:9092 --list
```

Initially, no topics will be listed (we haven't created any yet). We'll handle topic creation next.

Kafka 101: Topics, Partitions, and Producers/Consumers

Before we produce or consume data, it's important to understand how Kafka organizes it.

Apache Kafka is essentially a **distributed commit log**: producers append messages (events) to the end of a log, and consumers read those messages in order. A **topic** is like the name of a log or stream (for example, "user_signups" or "sensor_readings"). Topics are split into **partitions** to allow parallelism and scale.

- A **topic** is a category or feed to which messages are published. Think of it as the channel through which our data flows.

- **Partitions:** Each topic is divided into one or more partitions. A partition is an ordered, immutable sequence of messages – new messages are appended to the end. If a topic has multiple partitions, Kafka can distribute them across different brokers (servers). This allows multiple consumers to read in parallel and increases throughput. **Ordering** in Kafka is guaranteed **per partition** (not across the whole topic if there are multiple partitions).
- **Offsets:** Every message in a partition gets an incremental id called an *offset*. This is like a line number in the log. Consumers use offsets to keep track of where they are in the stream. For example, if a consumer has read up to offset 50 in partition 0, the next message it reads will be offset 51.
- **Producers and Consumers:** A **producer** is any application or service that sends data to Kafka (writes messages to a topic). A **consumer** is an application that reads data from a topic. In Kafka's publish/subscribe model, multiple consumers can independently read the same topic without interfering with each other, and at their own pace. Consumers often belong to **consumer groups** – a group of consumers that split the load by each one reading different partitions of a topic (ensuring each message in the topic is processed by only one consumer in the group).

For our pipeline: - We will create a topic (say, `streaming_users` or `api_events`) that our API data will go into. - We might give it a few partitions – for example, 3 partitions to simulate parallelism. (In a dev environment with one broker, partitions will still exist logically. In production with multiple brokers, partitions would be distributed among brokers.) - Our data ingestion script will be the **producer** writing to this Kafka topic. - Our Spark job will act as the **consumer**, reading from the Kafka topic.

Creating a Kafka Topic

Kafka doesn't require manual topic creation – by default, a topic is auto-created when a producer first publishes to it (if enabled). However, it's good practice (and in many setups required) to create topics explicitly with the desired number of partitions and replication factor.

We can create a topic using Kafka's CLI inside the Kafka container:

```
# Create a Kafka topic named "api_events" with 3 partitions and replication factor 1
docker exec broker kafka-topics --create \
  --topic api_events \
  --bootstrap-server localhost:9092 \
  --partitions 3 \
  --replication-factor 1
```

Output: If successful, you'll see a confirmation that the topic was created. We can verify by listing topics again:

```
docker exec broker kafka-topics --list --bootstrap-server localhost:9092
```

Now `api_events` should appear in the list. Great! Kafka is ready to receive data.

Why multiple partitions? Partitions allow Kafka to scale. With 3 partitions, Kafka can handle more throughput (producers can send to partitions in parallel, and consumers in a group can split the work by reading from different partitions). Also, if we had 3 Spark consumer tasks, each could be assigned

one partition's data. For our single Spark job, it will read all partitions, but under the hood Spark can parallelize reading from them. Even for development, it's useful to see how partitioning works. Each message will go to one partition (by default, Kafka will round-robin or use a hash of a key if provided).

What's a Kafka partition, in simple terms? If a Kafka topic is like a highway for data, partitions are like lanes on the highway. Cars (messages) in the same lane maintain order relative to each other. Adding more lanes (partitions) means more cars can travel in parallel, but each lane has its own sequence. When consuming, you typically read each lane separately (possibly with multiple readers) and then combine results as needed. Partitions make Kafka horizontally scalable while preserving order in each lane.

Now that we have a topic, let's start sending some data into Kafka via our API source.

Building the Data Producer (API to Kafka)

We need something to fetch data from an API continuously and feed it into Kafka. This could be a simple Python script or an Airflow task. The idea is to simulate an endless stream of events coming from an external source.

Our Example Scenario: Let's use the **Random User API** (<https://randomuser.me/>) as a stand-in for a source of user data events. Every time you call this API, it returns a JSON with a random user's details (name, email, etc.). In a real scenario, this could be any event source – e.g., an application emitting user sign-up events or an IoT device sending sensor readings.

We'll write a small **producer script** that does the following in a loop: 1. Call the API (get a new data point). 2. Send the result as a message to the Kafka `api_events` topic. 3. Wait a short interval (e.g., a few seconds) and repeat.

This loop will generate a continuous stream of data in Kafka.

Here's a simplified Python example using the `kafka-python` library (or you could use Confluent's Kafka library):

```
import json, time
import requests
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers=["localhost:9092"],
                          value_serializer=lambda v:
                          json.dumps(v).encode('utf-8'))

API_URL = "https://randomuser.me/api/"

while True:
    # 1. Fetch data from API
    try:
        response = requests.get(API_URL)
        data = response.json()
    except Exception as e:
```

```

        print(f"Error fetching data: {e}")
        time.sleep(5)
        continue

    # 2. Send data to Kafka
    producer.send("api_events", data)
    producer.flush() # ensure it's sent
    print("Sent data to Kafka:", data.get("results", [{}])[0].get("email"))
    # example field

    # 3. Sleep for a bit before next fetch
    time.sleep(5)

```

In this code: We configure a `KafkaProducer` to talk to our local Kafka broker. We fetch JSON from the Random User API, then we use `producer.send` to publish the JSON data to the `api_events` topic. We serialize the Python dict to JSON string bytes (via the `value_serializer`). We flush to ensure delivery (usually not strictly necessary each time but good for demo). We print an email from the user data just to have some visible output.

A few things to note for beginners: - We're sending the entire JSON response as one message. Kafka messages are just bytes – they don't care what format. We choose JSON so it's human-readable. In production, you might send more compact formats or even binary (e.g., Avro, Protobuf) for efficiency, but JSON is fine to start. - If the API returns multiple results in one call, we could also send each result separately. In the Random User API case, it returns one user by default. (We could increase the count to get multiple and loop through them.) - The `producer.send` is non-blocking; we call `flush()` to force it to send immediately and not batch, since we then sleep anyway. - The `bootstrap_servers` is pointing to `localhost:9092`. In Docker, since our producer might be running on the host or in another container networked to Kafka, we made Kafka available on localhost 9092 in the compose. This matches our Kafka's `KAFKA_ADVERTISED_LISTENERS` setting.

We can run this script directly on our machine (assuming we have Python and kafka-python library installed) or we could dockerize it. If using Airflow, we might instead incorporate this logic in an Airflow DAG (using a `PythonOperator` that runs periodically).

Using Airflow (Optional): We could create an Airflow DAG that triggers the above code every 5 seconds continuously. Airflow is usually for scheduled jobs, not infinite loops, but one approach is: - Use an Airflow **Sensor** or a loop of tasks to continuously run a small data fetch (maybe a task that sleeps and repeats). However, running an infinite loop inside Airflow isn't ideal. - Alternatively, schedule the task every minute and have each run fetch, say, 12 batches (one every 5 seconds) then finish. This simulates continuous ingestion but with managed chunks. - For simplicity, many streaming pipelines use a standalone script or a lightweight service for ingestion, outside of Airflow. Airflow might just monitor it or be used to start/stop it.

In our local demo, it might be easiest to run the producer script manually to populate Kafka. Once Kafka has some messages, we can move to Spark to consume them.

At this point: If you run the producer, you should see logs that data is being sent to Kafka. You can also open another terminal to consume messages for debugging:

```
# Read messages from the beginning of the topic for debugging (console consumer)
docker exec -it broker kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic api_events \
  --from-beginning
```

This will print out any messages in the topic (in their raw JSON form). If you see JSON lines appearing that match the API data, congratulations – you have a live stream of events in Kafka!

(Remember to stop the console consumer with Ctrl+C when done, or it will continue running, waiting for new messages.)

Spark Structured Streaming: Consuming and Processing Data

Now for the fun part – real-time processing with **Apache Spark**. We'll use Spark's Structured Streaming, which allows us to treat streaming data almost like a static DataFrame and use SQL-like operations.

Spark Setup: In our Docker Compose, we included a Spark service. There are a few ways to do this: - Use a single-node Spark (run Spark in local mode). Some Docker images (like `bitnami/spark` or the official Spark image) allow you to run `spark-submit` or start a Spark shell. - Set up a Spark master and worker containers (Spark standalone cluster). Then submit jobs to the cluster. - Use **PySpark** within a container (like running a Python script that uses Spark).

For simplicity, let's assume we can invoke Spark from the command line. We might write our Spark streaming logic in a Python script (say `spark_streaming.py`) and then run it with `spark-submit` inside the Spark container.

First, we need to ensure Spark can connect to Kafka. This means including the Kafka connector package when running Spark. The package coordinates are `org.apache.spark:spark-sql-kafka-0-10_2.12:<spark-version>`. If using Spark 3.3 or 3.4, we'd match the version. For example, if Spark is 3.4.0, we use `spark-sql-kafka-0-10_2.12:3.4.0`.

We also need the Kafka client library (though it often comes with that package). Typically, `spark-sql-kafka-0-10` will pull in the Kafka client jars. If not, specify `org.apache.kafka:kafka-clients:3.3.2` (for example) as well.

In `spark-submit`, you can use `--packages` to add these.

Writing the Spark Streaming Job

Our Spark job will do the following: 1. Connect to Kafka as a source, reading from the `api_events` topic. 2. Parse the incoming data (it will come in as bytes; we'll convert bytes to string, then parse JSON). 3. Perform any transformation. For demo, maybe select a few fields or add a timestamp. 4. Write the data out to a sink – here, Amazon S3. In local mode, we'll write to a local directory (which could be mounted to the container to see results). On AWS, this would be an S3 path (e.g. `s3://my-bucket/streamed-data/`). 5. Keep running as a streaming query, outputting continuously.

Let's outline a PySpark streaming script (`spark_streaming.py`):

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, timestamp_seconds
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType

# 1. Spark session with Kafka support
spark = SparkSession.builder \
    .appName("KafkaSparkStreamingDemo") \
    .getOrCreate()

# Optionally, set log level to reduce noise
spark.sparkContext.setLogLevel("WARN")

# 2. Define Kafka source DataFrame
df = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "broker:9092") \ # broker is the
Kafka container hostname in our Docker network
    .option("subscribe", "api_events") \
    .option("startingOffsets", "earliest") \           # read all existing
data first
    .load()

# The dataframe 'df' has schema: key (binary), value (binary), topic,
partition, offset, timestamp, etc.
# We only care about the value (the message body).
events_df = df.selectExpr("CAST(value AS STRING) as json_str")

# 3. Define schema of our JSON (for better performance)
json_schema = StructType([
    StructField("results",
        StructType([
# results is an array in randomuser, but let's assume 1 result
            StructField("gender", StringType(), True),
            StructField("email", StringType(), True),
            StructField("dob", StructType([StructField("age", IntegerType(),
True)]), True)
        ]), True),
    StructField("info", StructType([StructField("seed", StringType(),
True)]), True)
])

# Note: RandomUser JSON is nested; for brevity, I'm not capturing all fields,
just a few.

# Parse JSON string into a structured DataFrame
parsed_df = events_df.select(from_json(col("json_str"),
json_schema).alias("data"))
# Flatten it a bit:
flat_df = parsed_df.select(
```

```

    col("data.results.email").alias("email"),
    col("data.results.gender").alias("gender"),
    col("data.results.dob.age").alias("age"),
    col("data.info.seed").alias("batch_id")
)

# 4. (Optional) Add processing time or any transformations
from pyspark.sql.functions import current_timestamp
output_df = flat_df.withColumn("processed_at", current_timestamp())

# 5. Write to S3 (or local dir in our case)
query = output_df.writeStream \
    .format("parquet") \
    .option("checkpointLocation", "/tmp/spark_checkpoint") \
    .option("path", "/tmp/processed_data") \
    .outputMode("append") \
    .start()

query.awaitTermination()

```

Let's break down what this does in a beginner-friendly way:

- We create a `SparkSession`. In Structured Streaming, we use the same `SparkSession` as for batch.
- We use `spark.readStream.format("kafka")` to set up a streaming source from Kafka. We point it to the Kafka broker (note: inside the Spark container, it can reach Kafka by the service name `broker` if on the same Docker network; in other setups, use the appropriate host/port). We subscribe to our topic and set `startingOffsets` to `earliest` so we process from the beginning of the topic (for demo purposes; in a long-running job, you might use "latest" to only get new data).
- The `df` we get has the raw Kafka message. We cast the `value` from binary to string to get our JSON text. (We ignore the key and other metadata for now.)
- We define a schema for the JSON if we know it. This helps Spark decode the JSON faster and with correct types. The Random User API returns a nested JSON, but to keep it simple we pick a few fields (like email, gender, age, etc.). If we don't define schema, we could use Spark's `schema_of_json` to infer or just select with `get_json_object`, but explicit schema is clearer.
- We use `from_json` to parse the JSON string into a structured object (`data` column), and then we extract fields into a flatter DataFrame `flat_df`. For example, we extract email and age. (Note: The path `data.results.email` is pseudo-code; since "results" is an array in the JSON, we might need `data.results[0].email`. But for brevity assume one result.)
- We add a column `processed_at` with the current timestamp to mark when we processed that event.
- Finally, we write the stream out in **append** mode to Parquet files. We specify a checkpoint location – this is important! Spark uses checkpointing to store the state of the stream (like the offsets it has read from Kafka). This ensures fault-tolerance: if the job restarts, it knows where it left off and won't reprocess old data (ensuring **exactly-once** processing semantics in combination with how it writes output).
- The output path `/tmp/processed_data` in the container is where Parquet files will be written. In a real scenario, this would be an S3 URI (e.g. `s3://mybucket/stream-output/`). Actually, Spark can directly write to S3 if given an S3 path and proper credentials. For local testing, writing

to a mounted folder is fine. We could mount `./data` from host to `/tmp/processed_data` in the container to easily see files.

- We call `start()` to begin the streaming query, and then `awaitTermination()` so the program waits indefinitely while data streams.

Micro-batch vs Real Streaming: You might wonder, is Spark truly processing one event at a time? In Structured Streaming, by default it operates in micro-batches. It will group incoming events into small time intervals (the default can be a fraction of a second, or you can configure a trigger, say, 1 second). Each micro-batch is like a mini batch job where Spark will read whatever new messages arrived in Kafka, process them through the transformations, and write out a batch of Parquet files. This happens continuously. To us, it feels like a streaming job (we don't have to manually run each batch), but under the hood Spark is doing a series of small batch jobs quickly. The result is near real-time, and the benefit is we can use the rich Spark API (DataFrames, SQL, aggregations, etc.) with minimal changes from batch code.

What is Spark's micro-batching? In simple terms, Spark Structured Streaming collects incoming events for a short, fixed interval (like 1 second or 100ms) and processes them together. This is in contrast to processing each event one-by-one. Micro-batching adds a tiny bit of latency (you wait for the micro-batch interval), but it makes the system more efficient and easier to work with using Spark's existing APIs. From a beginner perspective, you can mostly write normal Spark code, and Spark takes care of continuously running it on new data. The micro-batch interval is usually small enough that you get results almost in real time (a second or two delay is often fine for many "real-time" needs).

Now, how do we run this Spark job? If we have a Spark container, we could use `spark-submit`. For example, if using the Bitnami Spark image, we might do:

```
docker exec -it spark-master spark-submit \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.4.0 \  
  /opt/spark-apps/spark_streaming.py
```

(This assumes our script is available in the container at `/opt/spark-apps/spark_streaming.py`, possibly via a volume mount or baked into the image. The `--packages` option pulls in the Kafka connector.)

If done right, Spark will start up, print a bunch of logs, and the streaming query will be running. You'll see log lines for each micro-batch as it reads data from Kafka and writes to files.

Verifying Spark Output

We configured the output to Parquet at `/tmp/processed_data`. If we mounted that to our host (or if we exec into the container), we should see files appearing as data flows. They might be in partitioned directories by some default (Spark might create a directory structure if we had partition columns; in our case we didn't partition by a column explicitly).

For example, after a minute of running, listing the output directory might show something like:

```
part-00000-...snappy.parquet
part-00001-...snappy.parquet
...
```

Each corresponds to one micro-batch's data. If we open one (using a Parquet viewer or Spark SQL), we'd see rows of data with the schema (email, gender, age, batch_id, processed_at).

For a simpler check, we could also have written to console for debugging:

```
# Instead of writing to Parquet, for debugging, do:
query = output_df.writeStream.outputMode("append").format("console").start()
```

This would print each batch of data to stdout. You'd see the rows in the logs. It's useful to verify things are working, but console output isn't feasible for long-term runs (it's just for testing).

Stop the Spark job by pressing Ctrl+C in the terminal where it's running (if you ran it interactively). In a real deployment, Spark jobs might run on a cluster where you'd stop them via Spark's job control or Yarn or AWS EMR interface.

We have now built the pipeline locally: data is flowing from the API to Kafka, then from Kafka through Spark to storage.

Let's talk a bit about how this would translate to a production AWS environment and how we orchestrate the pieces together.

Orchestrating and Productionizing the Pipeline

In our local setup, we manually ran a producer script and a Spark job. In a production scenario, you'd want these to be deployed services that run automatically, and you'd use orchestration/scheduling tools to manage them.

Orchestration considerations:

- **Starting the Producer:** We need a process continuously pulling from the API and writing to Kafka. In a simple case, this could be a small standalone application or microservice. To manage it, one might use a supervisor or an orchestrator. For example, you could containerize the producer script and deploy it in Kubernetes or as a long-running ECS task or even an AWS Lambda triggered on a schedule (though Lambda has time limits, so not ideal for infinite loop – but you could schedule a Lambda every minute that fetches and pushes one batch to Kafka).
- **Starting/Monitoring the Spark Stream:** The Spark streaming job should ideally run 24/7. On AWS EMR, you could run it as a step that doesn't terminate (using Spark Streaming in yarn cluster mode). Alternatively, AWS Glue Streaming jobs can run continuously using similar Spark Structured Streaming under the hood. Whichever service runs it, you'll need to ensure if it fails, it gets restarted. Tools like YARN on EMR can be configured to auto-restart failed apps, or you use an external scheduler (Airflow can trigger a Spark job and monitor it, for instance).
- **Airflow:** You may still use Airflow to orchestrate parts of this pipeline. For example:
 - An Airflow DAG to **trigger the Spark job** (though usually streaming jobs aren't re-triggered repeatedly, you might trigger it once or upon some event).
 - Or an Airflow DAG to manage the **producer**: e.g. using a loop to fetch data, as we discussed, or simply to ensure the producer container is up (Airflow could call a KubernetesPodOperator to ensure a deployment is running).
- Airflow is more naturally suited to batch jobs, but it can be used for streaming pipelines by acting as a

controller – for instance, a DAG that checks “is my streaming job alive? If not, start it” or one that handles governance tasks like rotating output data, or kicking off downstream batch processes on the streamed data.

Given Airflow was covered in the previous tutorial, here we’d clarify its role is different. In the earlier pipeline, Airflow orchestrated sequential tasks (extract then transform then load). In a streaming pipeline, there aren’t clear finish times for tasks; instead, Airflow might be used to orchestrate *auxiliary* tasks or manage deployments. Some teams schedule a streaming job to run daily but with a long duration, just to reset environment or incorporate new code, etc.

AWS Integration:

Let’s map our components to AWS services for a production deployment:

- **API Source:** If this were an internal API, it could be running on an EC2 or as a Lambda behind API Gateway. In our example we used a public API (randomuser.me). In production, you might not poll an external API continuously unless it’s your data source. But suppose you do (like pulling social media API for new posts). A more scalable approach might be to use a streaming ingestion service or the API’s webhook feature. AWS offers **Amazon Kinesis Data Firehose** or **DMS** for some sources, but for an HTTP API, you’d likely run a custom fetcher.
- **Kafka:** Use **Amazon MSK** to have a managed Kafka cluster. You would create a topic on MSK for your events. MSK takes care of running Kafka brokers, Zookeeper, etc. You’d just manage topics and data retention settings. Alternatively, AWS **Kinesis Streams** could replace Kafka – it serves a similar role (collecting streaming data). In our design, we stick to Kafka/MSK since that’s the chosen tech.
- **Spark:** Use **Amazon EMR** (Elastic MapReduce) which can run Spark jobs on a cluster of EC2 instances. EMR has support for long-running clusters or transient clusters. For streaming, a long-running EMR cluster running the Spark Streaming job is common. You’d submit your Spark job to EMR (using spark-submit or via steps API) and it would keep running. Another option is **AWS Glue Streaming Jobs** – Glue is a fully managed ETL service that runs Spark under the hood. Glue can run streaming ETL that reads from Kafka and writes to S3, similar to what we did. If you want to avoid managing servers, Glue might be attractive (you just provide the code and Glue runs it in its managed compute environment).
- **S3:** Simply an S3 bucket that you set up. You might partition the data by time (e.g. Spark could write into folders by date/hour if you add a partition column like event date). This makes downstream querying more efficient. Our example didn’t explicitly do that, but it’s a common enhancement.
- **Downstream:** Once data is in S3, you can use it further. For instance, **AWS Athena** (a serverless SQL query engine) can query the Parquet files in S3 directly – you’d just define a schema for them. This is great for ad-hoc analysis or feeding BI dashboards. Or you might load the data into a data warehouse (like Redshift) for more complex analysis or joining with other data. In the example pipeline prompt, we didn’t explicitly go beyond S3, but in production you likely will have a consumer of the S3 data.
- One pattern is to trigger an **AWS Lambda** when new files land in S3 (S3 event notifications) to perform some action, or to inform another system that new data is available.
- Another is to use **AWS Glue** (batch job) or **Airflow** to periodically crawl the S3 data and load into a database or run aggregations.

Security & Scaling: In a production pipeline, you’d also think about: - **Scaling Kafka:** number of brokers, partitions, retention period for messages (do you keep data in Kafka for a day or a week? Typically you might not keep it indefinitely if it’s stored on S3). - **Scaling Spark:** the EMR cluster size

(number of nodes) determines how many events per second you can process. The nice thing about Kafka + Spark is you can scale consumers to catch up if there's a spike – Kafka will buffer events. Just ensure your retention in Kafka is long enough to hold data until processed (maybe set retention to e.g. 24 hours so if Spark lags or is down, you have a day of cushion). - **Exactly-once processing:** It's worth noting that Spark Structured Streaming + Kafka can achieve *at-least-once* by default, but with checkpointing and writing to a fault-tolerant sink (like S3 or a transactional sink), you can get effectively exactly-once. Our pipeline writing to Parquet on S3 with a checkpoint and without any global aggregations is typically idempotent – duplicates are possible if something retries, but one can design around it (for example, include unique IDs and deduplicate downstream). - **Orchestration in AWS:** You might use Amazon Managed Airflow (MWAA) to schedule any batch parts or monitor the streaming job's health. Or use CloudWatch Alarms on the EMR job and Lambda to restart if needed.

To keep this beginner-friendly, the main point is: *the architecture we built locally translates to cloud services one-to-one*, and managed services on AWS can handle the heavy lifting of running Kafka and Spark. We focus on our code and data logic, and let AWS handle servers.

Cross-Component Handoffs

Let's narrate the journey of a single piece of data to reinforce understanding:

1. **Data generation (Producer)** – Suppose a new user signed up on our app, and we call our API or get a user object. Our producer code takes this event (user data in JSON) and sends it to Kafka topic `api_events`. Kafka immediately appends this event to one of the partitions of the topic (say partition 1). Now the event is durably stored in Kafka.
2. **Kafka buffering** – The event sits in Kafka. Kafka doesn't know or care who will read it; it just logs it with an offset (e.g., offset 120 in partition 1).
3. **Spark ingestion** – Spark Streaming is subscribed to the `api_events` topic. Spark periodically checks Kafka for new messages (this happens under the hood when we call `.readStream` and start the query). When our new event is available, Spark will fetch it (along with any other new events in that micro-batch window). Spark keeps track of the last offset it read in each partition – so let's say before it had read up to offset 119, now it sees offset 120 is available, it will read that.
4. **Processing** – Spark takes the raw JSON string of the event, parses it into structured columns (email, age, etc., as we coded). It might do transformations like filtering out certain events or adding a timestamp.
5. **Output to S3** – Spark then writes the transformed data out to S3. This could mean adding a new row in a Parquet file in an S3 folder. Because we're in micro-batch mode, Spark might accumulate a few events and then write them together as one Parquet file once a threshold or time is met. This file is now stored in S3.
6. **Downstream consumption** – Now that the event's data is in S3, other systems can use it. For instance, an analyst can query it using Athena, or a daily batch job might aggregate all events per day for a summary report, or a dashboard might refresh to include the new data. In our pipeline scope, we stop at S3, but it's good to see that the pipeline made the data available for *use*, which is the ultimate goal.
7. **Acknowledge & offset commit** – As Spark processed and wrote the data, it will commit the offsets (in checkpoint metadata or to Kafka if using Kafka's offset commit). This means Spark records "I have processed up to offset 120 in partition 1." If Spark restarts, it will know to resume from 121 onward. Kafka's role is done for that message – it will keep it for a configured retention time in case any other consumer needs it or in case our Spark job fell behind and needed to reread. After retention time, Kafka will drop it to save space.
8. **Continual loop** – The producer keeps sending events, Kafka keeps logging them, Spark keeps reading and writing results. This loop can run continuously, delivering data in (near) real-time.

Throughout this, notice how each component is **decoupled**: - The producer doesn't send directly to Spark or S3; it only knows about Kafka. It doesn't need to know if Spark is slow or down – Kafka will buffer. - Spark doesn't query the API or talk to the producer; it only reads from Kafka. If the API floods Kafka with data faster, Spark can scale out (with more executors) to catch up or process in parallel thanks to partitions. - S3 storage is separate from Spark's processing memory. Once data lands in S3, Spark's job for that micro-batch is done. If a downstream process needs to transform data from S3 to a warehouse, it's another pipeline or job, which could be orchestrated by Airflow after certain intervals or triggered by S3 events.

This decoupling is what makes event-driven architectures robust and scalable. Each component can fail or scale independently: if Spark goes down for an hour, Kafka will retain the data; when Spark comes back, it resumes. If Kafka backs up because Spark can't keep up, we can scale Spark or add more consumers without touching the producer.

Transitioning from Docker to AWS (Summary)

To deploy our pipeline on AWS in a production-ready way, here's a quick checklist of steps (this is more for your understanding rather than an exercise we do now):

1. **Set up Amazon MSK**: Create a Kafka cluster on AWS (MSK). Configure the topic (number of partitions, etc.) – you can do this with Kafka tools pointed at MSK's bootstrap servers.
2. **Deploy Producer**: Package the data ingestion code (API polling) into a container or AWS Lambda. If using Lambda, ensure it can send to MSK (there's VPC considerations). Alternatively, deploy a small EC2 or ECS service running that Python loop. Ensure it's robust (e.g., it should handle API errors, and you might want multiple instances if high availability is needed).
3. **Spark on EMR**: Create an EMR cluster with Spark. Upload the Spark streaming script to S3 or cluster. Run it as a step or deploy it as a long-running application (EMR has a cluster mode where you SSH and run `spark-submit`, or use Livy server, etc.). Configure checkpoints to a durable path (e.g., an S3 location) so state is preserved if the driver restarts. Alternatively, set up an AWS Glue Streaming Job with the same code, pointing to MSK and S3, and let Glue run it continuously.
4. **S3 Bucket**: Create an S3 bucket for output data. Set proper permissions (the Spark EMR cluster or Glue needs write access to that bucket). Consider partitioning scheme for files (maybe partition by date).
5. **Orchestration/Monitoring**: Use Airflow (MWAA) or AWS Step Functions to coordinate if necessary. For example, Airflow could be used to deploy the Spark job (though often you'd just start it manually or via EMR schedule). At minimum, set up CloudWatch alarms:
6. For MSK (e.g., if consumer lag grows too large, alert).
7. For EMR/Glue (if the job stops or errors out, alert).
8. For the producer (if it's a Lambda, use CloudWatch metrics or if EC2, maybe a heartbeat). These ensure you know if something breaks.
9. **Cost and Scale**: Ensure you right-size your resources. Kafka MSK and EMR cost money continuously. If the data rate is low, maybe a single node MSK and a small Spark cluster (or even using AWS Kinesis & AWS Lambda for simple transformations) could suffice. If data is big, ensure enough partitions and Spark executors for parallelism.

One huge advantage of AWS is that a lot of heavy lifting can be managed: MSK handles Kafka maintenance, EMR can autoscale Spark to some extent, and S3 scales automatically for storage.

Conclusion

In this tutorial, we built an end-to-end *event-driven* data pipeline that mirrors what you'd find in many real-world data engineering scenarios. We started by containerizing our environment with Docker, ensuring we could run Kafka and Spark locally without hassle. Then we created a continuous data generator (simulating an API source) and used **Kafka** to ingest and buffer those events. We explained how Kafka topics, partitions, and offsets work – demystifying the concept of a distributed commit log for beginners.

On the processing side, we introduced **Spark Structured Streaming** as a powerful yet approachable way to handle streaming data. We showed how a Spark job can read from Kafka in micro-batches, apply transformations using high-level DataFrame APIs, and write the results to **Amazon S3** in real-time. Along the way, we discussed what micro-batching means (processing data in small time slices) and why it provides a nice balance between performance and real-time latency.

Crucially, we maintained a **narrative of how each component hands off to the next**: the API producer hands data to Kafka, Kafka hands data to Spark, and Spark hands off the processed data to S3. Each component is specialized – Kafka decouples ingestion and consumption, Spark handles complex processing, S3 provides durable storage. This decoupling makes the pipeline scalable and fault-tolerant. We also touched on **orchestration** – using tools like Airflow to schedule or monitor these processes, and how in a continuous pipeline the role of orchestration shifts more towards keeping things running rather than one-time scheduling.

By comparing our local Docker setup with an AWS deployment, we demonstrated how one can **develop locally and then transition to cloud**. The pipeline we built locally can be almost directly mapped to AWS managed services (MSK, EMR, etc.), which means you can prototype on your machine and then scale up in production.

For someone early in their data engineering journey, you've now seen a full example of streaming data integration. You can take this foundation and extend it: for instance, add a real analytics step by querying the S3 data with a tool or loading it into a warehouse, or incorporate schema registries for Kafka for stricter data contracts, or use multiple topics and streaming joins for more complex pipelines. The key principles will remain the same:

- **Don't assume prior knowledge** – break down the problem (we hope we did – explaining each concept like Kafka's role, Spark's approach, etc.).
- **Use the right tool for each job** – and let them do what they're good at (Kafka for streaming transport, Spark for processing, etc.).
- **Automate and orchestrate** – ensure the pipeline can run reliably (Docker for local, Airflow/Cloud for prod).
- **Think production** – even in a toy example, we considered checkpoints, scaling, and monitoring, which are all critical when this runs for real.

We encourage you to experiment with the code snippets provided. Try running the Docker setup, tweak the producer to send different data, alter the Spark transformation (maybe compute a running average or filter certain users), and see the results live. Streaming data opens up a world of possibilities for real-time analytics and reactive applications. With the knowledge gained here, you're well on your way to building your own real-time data pipelines.

Happy streaming and happy data engineering!
